

Agentory

Lead Retrieval Architecture — end-to-end audit

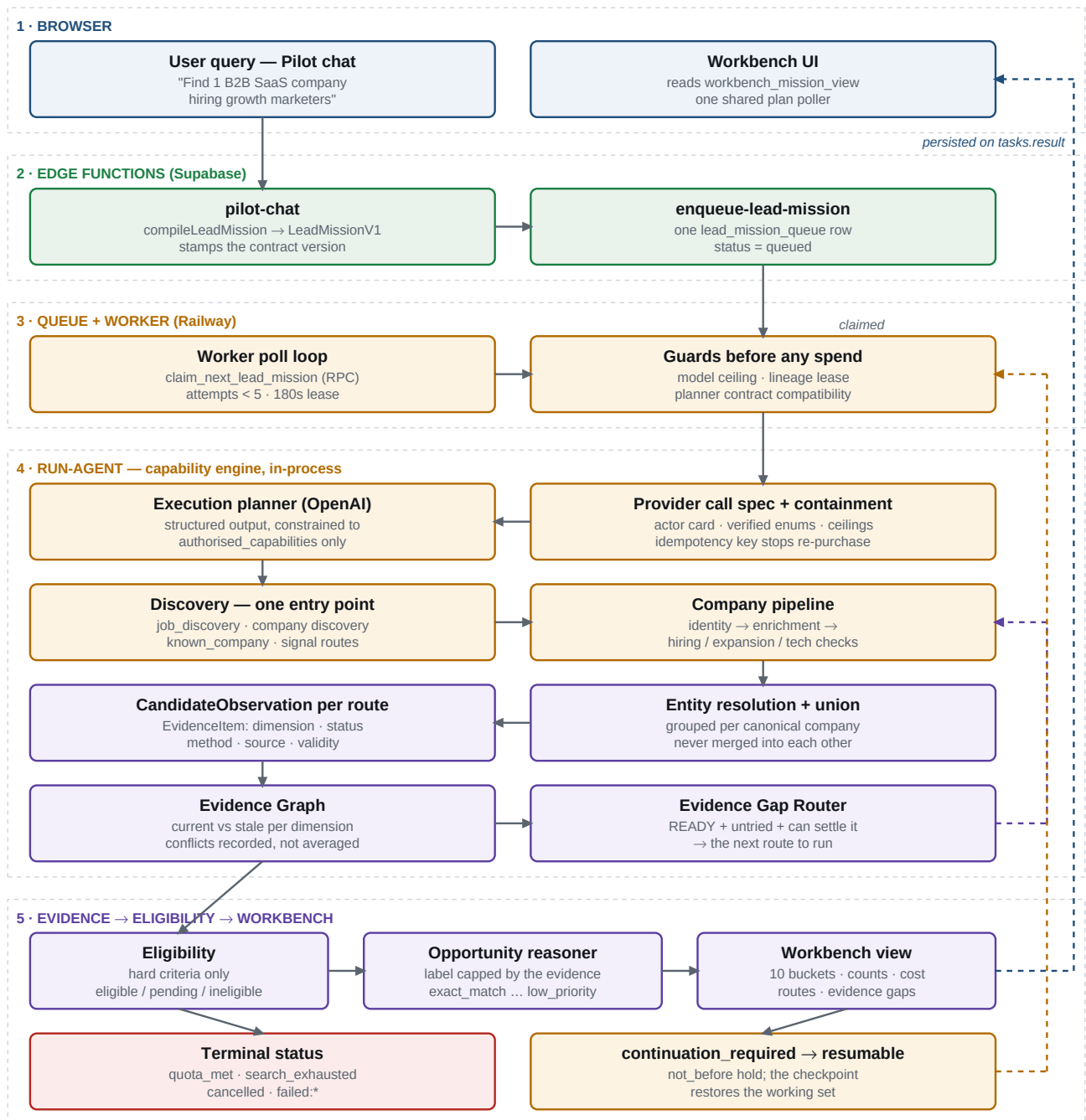
Question	From a sentence typed in Pilot to a row rendered in the Workbench: what runs, in what order, what it is allowed to spend, and where the architecture is still unproven.
Date	20 September 2026
Repository	agentory-main-local · branch feat/lead-mission-v2-worker · HEAD 7826f88d (local)
Verified against	Source at HEAD, the local Supabase stack, and a live mission run end to end on 20 September
Production	Not contacted. Every runtime observation is from the local stack.

The one-sentence summary

The pipeline is **evidence-first**: discovery finds companies, verification proves claims, and evidence gaps choose the next route. Nothing becomes a qualified lead because a model said so — a claim is settled by a provider field with provenance, or it stays pending. The spine is complete and runs end to end. What it cannot currently do is *finish*: with no Apify credential in the local environment and one verification route still without a proven provider, missions terminate honestly at `search_exhausted` with zero qualified companies.

Workflow

Solid arrows are the forward path. The three dashed rails on the right are what make this iterative rather than a single pass. **Purple**: the verification loop — the gap router picks a route that re-grounds an unsettled claim, and the pipeline runs it. **Amber**: the continuation loop — a slice ends short, the queue re-claims it after a hold, and the checkpoint restores the working set. **Blue**: the result, persisted on `tasks.result` and read by the Workbench.



Stage by stage

#	Stage	Where	What it guarantees
1	User query → mission	pilot-chat → compileLeadMission	The sentence becomes a LeadMissionV1. Precedence is fixed: the explicit query overrides the model proposal, which overrides the Company Brain, which overrides defaults. The Brain may refine but never widen. The mission is stamped with a contract version.
2	Mission → queue	enqueue-lead-mission	One row in lead_mission_queue, status queued. The request carries the compiled mission, so the queue is the unit of work and the mission is immutable from here.
3	Queue → worker	claim_next_lead_mission RPC	A single claimant: attempts < 5, a 180-second lease, and rows whose lineage is cancelled or terminal are never claimed. The worker runs run-agent's handler in-process, so its own environment governs the guards.
4	Guards before spend	run-agent	Model-spend ceiling, workspace membership, lineage lease, and planner-contract compatibility — all evaluated before a task row exists, so a refusal leaves no orphan and buys nothing.
5	Execution plan	gptExecutionPlanner	OpenAI structured output, constrained to authorised_capabilities. A provider failure throws rather than substituting a deterministic plan — a guessed plan spends real money.
6	Discovery	one entry capability	job_discovery, startup/general company discovery, known_company_resolution, or a signal discovery route. Mutually exclusive: a mission has one way in.
7	Company pipeline	identity → enrichment → verification	Identity is resolved before anything consumes a company. Every provider call is bounded by its actor card's verified enums and ceilings, and carries an idempotency key so a continuation cannot re-buy a page.
8	Observation → evidence	CandidateObservation	Each route's statement about one company, with the facts as EvidenceItems carrying dimension, status (proven / plausible / disproven / unknown), method, source and validity. GPT never writes an observation.
9	Resolution + graph	entity resolution → Evidence Graph	Observations are grouped under a canonical company, never merged into each other. The graph keeps current versus stale per dimension and records conflicts instead of averaging them.
10	Gap → next route	evidenceGapRouter	For each unsettled claim: is a route READY, untried for this company, and able to turn its result into a canonical decision? Only then is it executable. This is what stops 'buy another discovery page'.
11	Eligibility	candidateEligibility	Hard criteria only. A proven item may pass or fail a rule; a plausible one never decides. Missing evidence yields pending, never ineligible.
12	Label	opportunity reasoner	exact_match / strong_opportunity / worth_considering / low_priority, capped by the evidence actually cited. A reasoner that cites nothing real is dropped a level.
13	Workbench view	buildWorkbenchMissionView	Ten buckets, counts, routes, evidence gaps and cost, persisted on tasks.result and read by the UI.

Invariants the architecture enforces

Invariant	How it is enforced	Proven
Discovery finds, verification proves	Separate capabilities; the gap router picks the next route from unsettled claims	Yes — canary 45dc045f: 11 verifications ran while job purchases stopped at 2
A claim is settled by evidence, not by a model	EvidenceItem carries method and source; model_extraction cannot alone prove a claim	Yes — unit and mutation tested
Missing evidence never becomes a rejection	Eligibility returns pending when nothing proven speaks	Yes
No page is bought twice	Idempotency key per provider call; the router marks a route tried per company	Yes — no duplicate keys in the live canary
A refused actor is remembered	Containment records refused_policy rather than retrying each slice	Yes
Continuation reuses the plan	The frozen plan is held unless a validated amendment occurs	Yes
Checkpoint preserves the working set	Bounded snapshot restores identity, provenance, observations and grounding	Yes — round-trip tested after the found_by regression
Spend is bounded before it happens	Ceilings on the actor card, the mission, and the workspace's model budget	Yes
The user's criteria are never rewritten	Brain may refine, never widen; compiler precedence is fixed	Yes

Where the architecture is incomplete

Gap	Consequence	Severity
No funding-stage verifier is proven. Four actors were probed on 19–20 September: datahyena takes no company input, memo23 gives a round type but no date or complete history, enrich-crm is out of vendor credit, and the two company-record actors each satisfy only half the requirement.	The funding_stage claim can never be settled, so any mission with a stage criterion holds a permanent pending. The deterministic SEED-1 decision module is written and tested; it has nothing to consume.	HIGH
The employees/team-composition actor is opt-in and disabled.	first_in_function cannot be proven. Zero employee results are correctly never treated as proof, so the claim stays pending rather than resolving wrongly.	MEDIUM
No Apify credential in the local environment.	Discovery returns nothing locally; missions terminate search_exhausted with zero companies. Correct behaviour, but it means the full pipeline has never been observed locally with real rows.	MEDIUM
The lead pipeline has never run end to end against a live model.	The planner was exercised locally through a stub. Model identifiers and structured output are separately live-verified, but the composed path is not.	MEDIUM
Qualification depends on web evidence stating a fact in words.	A company whose site never says it sells software to businesses stays pending even when it plainly does. This is deliberate conservatism, and it is also why live canaries return zero qualified.	BY DESIGN

What the live run actually did

A full mission was run on the local stack on 20 September with the planner stubbed and no provider credentials, to exercise the spine without spend.

Step	Observed
Queue	Row 2e213c08 claimed by worker 86cd38e5
Guards	Spend ceiling passed only after both MODEL_SPEND_CEILING_USD and MODEL_SPEND_PERIOD_DAYS were set; a hand-written mission was refused until recompiled with a contract version
Task + lineage	bb5b9e2e created, lineage opened
Plan	2 planner calls, 0 tokens, \$0.000000 — the stub
Discovery	entry job_discovery; allowed providers limited to the mission's own graph
Rows	0 companies — no Apify token, so nothing was returned
Continuation	continuation_required → 2-minute hold → re-claimed as the next slice
Terminal	search_exhausted, queue complete
Workbench	View built with counts, routes, evidence_gaps and cost; all zeros, correctly

Cost model

Every provider call is priced from its actor card before it runs, and metered after. Model calls enter `Lead_model_calls` with tokens and an estimated cost; provider runs are bounded by the card's ceilings and the mission's own budget. The observed prices from the September probes: LinkedIn profile \$0.005 per profile; LinkedIn company details \$0.0035 per company found, with not-found rows free; Firecrawl roughly one credit per page; the funding actor \$0.07 per result after a 56% rise on 12 September.

The one structural gap: onboarding's Apify call passes no maximum charge, so it trusts the actor's own pricing rather than bounding the run.

Verdict

The spine is sound.	Queue, worker, guards, plan, discovery, observation, resolution, graph, gap routing, eligibility, labelling and the Workbench view all run, in order, with the invariants enforced and tested.
The evidence model is the strong part.	Provenance, staleness, conflicts and the proven/plausible distinction are all real and load-bearing, not decoration. The conservatism is correct even though it currently produces zero qualified companies.
The weak point is provider coverage, not design.	Two claims have no provider that can settle them, and the local environment has no discovery credential. Neither is an architectural fault; both stop the pipeline from finishing.
Before calling it production-ready	Prove one funding verifier, enable a discovery credential in a controlled environment, and run the composed path once against a live model with a hard spend cap.